

LLMTailor: A Layer-wise Tailoring Tool for Efficient Checkpointing of Large Language Models

Minqiu Sun
University of Delaware
Newark, United States
mqsun@udel.edu

Xin Huang
RIKEN Center for Computational
Science
Kobe, Japan
xin.huang@a.riken.jp

Luanzheng Guo
Pacific Northwest National
Laboratory
Richland, United States
lenny.guo@pnnl.gov

Nathan R. Tallent
Pacific Northwest National
Laboratory
Richland, United States
Nathan.Tallent@pnnl.gov

Kento Sato
RIKEN Center for Computational
Science
Kobe, Japan
kento.sato@riken.jp

Dong Dai
University of Delaware
Newark, United States
dai@udel.edu

Abstract

Checkpointing is essential for fault tolerance in training large language models (LLMs). However, existing methods, regardless of their I/O strategies, periodically store the entire model and optimizer states, incurring substantial storage overhead and resource contention. Recent studies reveal that updates across LLM layers are highly non-uniform. Across training steps, some layers may undergo more significant changes, while others remain relatively stable or even unchanged. This suggests that selectively checkpointing only layers with significant updates could reduce overhead without harming training. Implementing such selective strategies requires fine-grained control over both weights and optimizer states, which no current tool provides. To address this gap, we propose LLMTailor, a checkpoint-merging framework that filters and assembles layers from different checkpoints to form a composite checkpoint. Our evaluation indicates that LLMTailor can work with different selective checkpointing strategies and effectively reduce checkpoint size (e.g., 4.3 times smaller for Llama3.1-8B) and checkpoint time (e.g., 2.8 times faster for Qwen2.5-7B) while maintaining model quality.

CCS Concepts

• **Computer systems organization** → **Dependable and fault-tolerant systems and networks; Availability**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Checkpoint, Large Language Model, I/O optimization

ACM Reference Format:

Minqiu Sun, Xin Huang, Luanzheng Guo, Nathan R. Tallent, Kento Sato, and Dong Dai. 2025. LLMTailor: A Layer-wise Tailoring Tool for Efficient Checkpointing of Large Language Models. In *Workshops of the International*

Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25), November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3731599.3767515>

1 Introduction

Originally developed in high-performance computing (HPC) to protect long-running scientific simulations, checkpointing has become the primary safeguard against failures in large-scale, high-cost LLM training. By periodically writing the complete model state, which includes weights, optimizer parameters, and other metadata, to persistent storage, the system can reload the most recent snapshot after a failure and resume training with minimal loss of progress [9].

Since current checkpointing uniformly saves all LLM layers, it introduces significant I/O overhead. As training scales up, the checkpoint-related overhead can account for 12% of total training time and can rise to as much as 43% [18]. We believe this ‘saving the entire LLM states’ approach might not be efficient, as many studies show that layers in LLMs are not updated at the same pace.

Many recent works have highlighted the imbalances across layers of LLMs. Jawahar et al. [14] show that different layers of large language models encode distinct types of linguistic information. Phang et al. [23] observe that the lower and higher layers of fine-tuned RoBERTa and ALBERT models are dissimilar. Zhou et al. [33] propose the *PL_Alpha_Hill* metric, demonstrating that different layers train at different speeds when using optimizers such as SGD and Adam. Collectively, these findings indicate that the existing checkpointing mechanisms, which indiscriminately save the complete model, are not optimal. It is then natural to explore the idea of keeping only part of the model during training. For example, checkpointing half of the layers at a time, and merging 2 checkpoints into a complete one. Clearly, doing so requires reconstructing a complete training state from multiple partial checkpoints, which in turn demands fine-grained manipulation of individual layers, including both weights and optimizer states. To our knowledge, no existing tool offers this capability. The closest one, MergeKit [10], merges the checkpoints only on weights and omits optimizers, making the recovery of training impossible. This is clearly not acceptable for checkpointing, as it is designed for training.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SC Workshops '25, St Louis, MO, USA*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1871-7/2025/11

<https://doi.org/10.1145/3731599.3767515>

In this study, we develop LLMTailor, a tool that merges both weights and optimizer shards to generate a fully resumable “Frankenstein” checkpoint, which can be used to continue training. With LLMTailor, we can investigate the feasibility and overhead of partial checkpointing during training.

To validate our design, we conduct extensive experiments to determine whether 1) partial checkpointing with LLMTailor reduces storage and checkpoint time, 2) the checkpoint made by LLMTailor will negatively impact the model quality; On Llama-3.1-8B, LLMTailor reduces total checkpoint size by 4.3 times; on Qwen-2.5-7B, we reduce the ratio of checkpoint time to end-to-end training time by 2.8 times. In another simple use case, we reduce the checkpoint size by half while maintaining model accuracy. The contributions of this study are threefold:

- (1) We present LLMTailor, an effective tool that assembles a resumable “Frankenstein” checkpoint from parts of multiple checkpoints while maintaining the model performance.
- (2) We demonstrate that partial checkpointing with LLMTailor can substantially lower storage requirements and checkpoint time in LLM training.
- (3) We show that LLMTailor incurs only a small amount of overhead when resuming training from a merged checkpoint.

The rest of the paper is organized as follows. In §2 we discuss relevant backgrounds of LLMs. In §4 we describe the architecture of LLMTailor in detail. We present the extensive experimental results in §5. §6 discusses the closely related work. In §7 we present our conclusions and discuss future work.

2 Background and Motivation

Large language models (LLMs) have demonstrated remarkable power in various domains [5–7, 21]. LLM training, like other neural networks, involves a forward pass, loss computation, a backward pass, and parameter update. Here, we review the key fundamentals that motivate our layer-wise checkpointing: (i) the structural components of LLMs, (ii) the organization of optimizers, and (iii) the distributed frameworks such as DeepSpeed used for scaling.

2.1 Structure of LLMs

Figure 1 uses the Llama3-8B model as an example [2]: tokens are first embedded from a number to a vector, then propagated through 32 consecutive transformer layers. Then, after layer normalization to eliminate internal covariate shift, the hidden representations are projected back to the vocabulary space to produce logits in the `lm_head` layer; finally, a softmax yields output probabilities. Note that not all models include a separate `lm_head`; in smaller models, this layer is often weight-tied to `embed_tokens` to reduce parameter count [24].

Each transformer block contains two layer-normalization sub-layers that stabilize activation statistics. Between them, the self-attention module performs dynamic context aggregation, the defining operation of the Transformer architecture. The feed-forward network (FFN) then applies a non-linear transformation to each token representation: it expands the hidden dimension, activates it (e.g., with SwiGLU), and projects it back to the original size, thereby enriching the model’s ability to capture complex patterns. In the

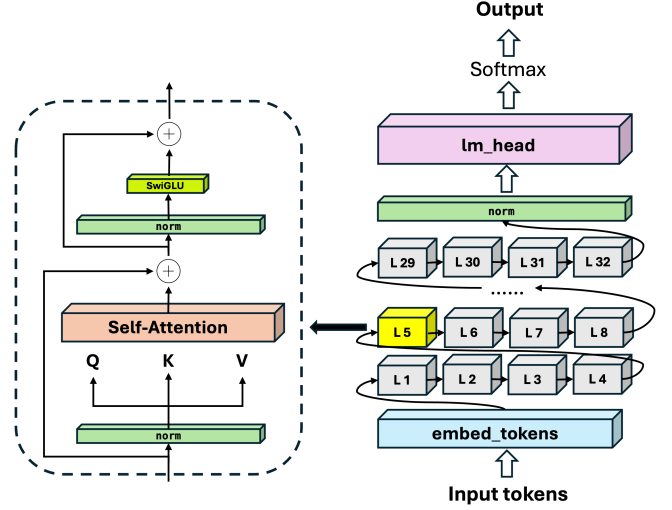


Figure 1: The layer-wise structure in a Llama3.1-8B model.

right panel of the figure, each box represents a trainable layer; collectively, these layers comprise the large language model.

2.2 Structure of Optimizers

The parameters are continuously updated to minimize the training loss, and this update process is applied by an optimizer, such as stochastic gradient descent (SGD) and Adam. Today, optimizers in training LLMs are often from the Adam family rather than plain SGD, due to their strong ability to accelerate and stabilize convergence during LLM training. As Equation 1 shows, Adam maintains two running momentum estimates by continuously accumulating gradients into two auxiliary tensors m and v (the first- and second-order momentum). So, the optimizer has double the parameter size as the model weights due to the two momentum terms.

$$W_i(t) \leftarrow W_i(t-1) - lr(t) \cdot \frac{m}{\sqrt{v} + \epsilon}. \quad (1)$$

Figure 2 sketches the layout of the optimizer file in a checkpoint. During the update stage, all model parameters are flattened into two parameter groups, disregarding the model’s hierarchical or sub-layer structure, to improve computational efficiency. To preserve adequate optimization control, each group is assigned its own hyperparameters (e.g., learning rate, weight decay). Grouping at this coarse granularity rather than per tensor strikes a practical balance between efficiency and flexibility. In Figure 2, the two groups are divided by different weight decay. This choice is because AdamW decouples weight decay from the loss gradient; it is standard to apply decay only to weights, while excluding biases and normalization parameters. Shrinking the latter can harm stability without offering meaningful regularization. Accordingly, one group contains all biases and normalization parameters (with zero weight decay), and the other contains the remaining weights (with nonzero weight decay).

Checkpointing safeguards training continuity by periodically snapshotting every component whose state evolves during optimization. The most obvious target is the model itself. In addition, the

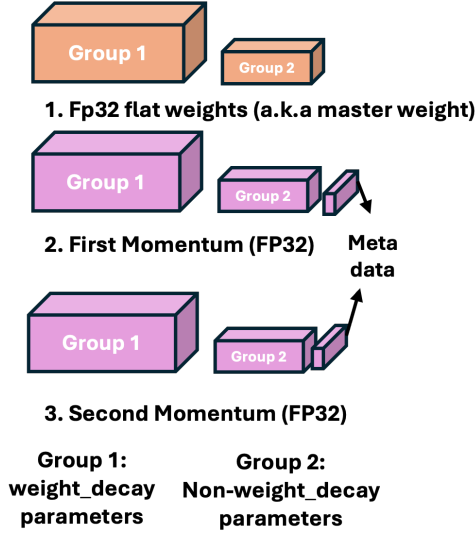


Figure 2: The AdamW optimizer used in a Llama3.1-8B model.

Adam optimizer relies on accumulated gradient moments to ensure precise and stable updates; omitting these optimizer states from the checkpoint can lead to large spikes or noisy updates when training resumes. In addition to the momentum terms, the file contains an FP32 master weight tensor of the same size. This duplication is required for mixed-precision training: forward and backward propagation run in FP16/BF16 to exploit tensor-core throughput and reduce memory-bandwidth demands, while FP32 master weights and FP32 momentum estimates are retained for numerical stability. Consequently, including some remaining configuration files, a single checkpoint must store at least $7 \times$ the size of the FP16/BF16 model itself.

2.3 Distributed LLM training under Deepspeed

Since LLMs have now grown to unprecedented scales, this footprint makes distributed frameworks such as DeepSpeed’s ZeRO vital to LLM training. It can shard optimizer states, gradients, and parameters across data-parallel processes, dramatically reducing the per-GPU memory footprint [27, 28]. For example, in the ZeRO3 stage, each rank holds only a shard of the model parameters and optimizer states. During forward/backward passes, ZeRO-3 all-gathers the needed parameter shards just-in-time for a layer, computes them, and then re-shards them. In distributed training checkpoint systems, optimizer states are saved as shards: each GPU writes only its own shard to reduce overhead. In contrast, to ensure that it can be used for reasoning at any time, the model weights are typically stored as a single consolidated file. During recovery, every optimizer shard is loaded onto its corresponding GPU, ensuring that the training state is restored correctly.

3 Brief Introduction about Mergekit

Mergekit [10] is a lightweight command-line toolkit that lets users compose new language-model checkpoints from two or more existing ones with only a short YAML recipe. Internally, it loads each source model, matches tensors by name, and applies a user-selected

merge rule. To use it, users first select a merge method, such as *linear blending*, *SLERP*, *passthrough copy*, or *LoRA-fusion*. Next, users need to write a recipe YAML that lists source models and specifies layer-selection rules. And then run the CLI to emit a new model assembled according to these rules. After that, the resulting Frankenstein model can be loaded directly by standard PyTorch or Hugging Face runtimes. In practice, users most often apply it to tasks like model capability fusion and domain adaptation.

Although mergekit supports several merging strategies, layer-wise checkpoint merging relies on the passthrough method, which splits and recombines source models by layer. Because MergeKit works only with weight files and therefore avoids both back propagation and retraining, it cannot support full checkpoint merging for the following reasons:

- (1) Optimizer states are ignored. Mergekit merges only model weights, omitting optimizer files that are essential for resuming training.
- (2) Auxiliary layers are excluded. It manipulates transformer layers only, leaving out large auxiliary layers such as token embeddings and the prediction head.
- (3) Configuration files are not handled. Mergekit does not provide support for merging or editing checkpoint configuration files.

Given the popularity and practicality of MergeKit, we want to adopt its YAML-driven interface and extend its functionality to handle full checkpoints, including optimizer states, auxiliary layers, and configuration metadata.

4 Design and Implementation

In this section, we introduce LLMTailor, which constructs resumable training checkpoints by composing model layers (and associated optimizer states) from multiple checkpoints without changing the way mergekit is used. The key design insights behind LLMTailor are detailed below.

4.1 Construct Separable Optimizers in Checkpoint

Because MergeKit cannot merge optimizer states from different checkpoints, our LLMTailor extends it to enable this functionality. The main challenge arises from the optimizer’s intrinsic structure: unlike model weights, which follow a layer-wise hierarchy, optimizer files store flattened tensors that are difficult to split or merge. The only natural partition point in these files is the parameter group. We therefore reconstruct the parameter groups to mirror the model’s layer-wise organization while preserving the original weight-decay settings. Specifically, each transformer layer is divided into two groups: one containing tensors exempt from weight decay and the other containing the remaining tensors. Since auxiliary layers contain exclusively either weight-decay or non-weight-decay parameters, they are assigned to a single parameter group. Since the ordering of such parameter groups is consistent across different LLMs, knowing only the total number of transformer layers and whether weight tying is applied by reading the configuration file is sufficient to determine the parameter group index of each layer in the optimizer file. As a result, the number of parameter groups increases from two to $2L + x$, where L is the number of transformer

layers, x is the number of auxiliary layers. Figure 3 illustrates the transformation of a 16-layer, 2-group model into a 35-group model.

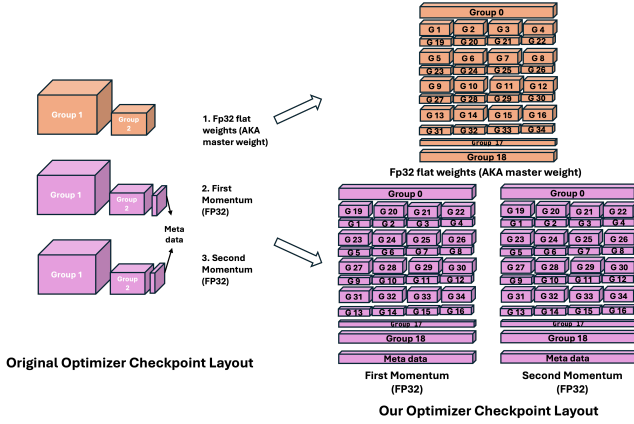


Figure 3: Reconstruct the parameter groups in the optimizer before training.

The checkpoint’s structure must match the model in GPU memory during training or before failure; otherwise, the checkpoint is unusable. As a result, we perform this regrouping before training begins. During training, the master weights are automatically organized into the same 35 groups, ensuring that the optimizer file stored in each checkpoint has a uniform, group-aligned structure. Next, we merge and assemble a new checkpoint by tracking the indices of the two parameter groups associated with each transformer layer.

This redesign renders the optimizer files separable, while preserving the original weight decay configuration. Because neither parameters nor hyperparameters are altered, the training procedure and final results remain unchanged; the only additional cost is a small amount of computational overhead. This helps solidify the basis for tailoring checkpoints.

4.2 Merge Optimizers

Supporting arbitrary assembly of layers from multiple checkpoints, especially in the optimizer files, is a key objective. Since MergeKit already handles weight merging, we focus here on our implementation of the optimizer file.

LLMTailor first parses a YAML specification that lists the base model, the source layers with their corresponding checkpoints, and the target positions of those layers in the new model. The fixed parameter-group structure lets us locate each layer and its associated groups. Figure 3 illustrates the default ordering of parameter groups: the first group stores the normalization layer, followed by the non-weight-decay segments of the transformer layers, then the embedding layer and the optional `lm_head`, and finally the weight-decay segments of each transformer layer. When a transformer layer is selected, LLMTailor automatically indexes and copies all its parameter groups, inserting them into the user-defined position in the new model.

Unlike the unified model-weight file, each parameter group is uniformly sharded across GPUs, and the corresponding checkpoints are stored in separate files because of distributed training. The

largest overhead in LLMTailor is the I/O overhead of loading up to $N \times (L + 3)$ optimizer files and writing up to N files, where N represents the total number of GPUs. To ensure the correctness of the resumed checkpoint, we keep the order of loading and writing. In addition, we leverage multiprocessing to accelerate this process. We employ Python’s `ProcessPoolExecutor` to parallelize shard loading across multiple CPU cores, enabling concurrent decompression and deserialization of large ZeRO-3 optimizer files, which significantly reduces overall I/O latency.

4.3 Split Auxiliary Layers

Since mergekit focuses solely on model weight merging, it retains the base model’s `norm`, `embed_token`, and `lm_head` for vocabulary compatibility. However, the latter two contain the entire vocabulary and thus occupy far more space than individual transformer blocks. We therefore adjust the merge plan to split and merge these auxiliary modules explicitly. In LLMTailor, the only extra requirement is that users list these three modules in the YAML recipe.

4.4 Manipulate Configuration Files

Metadata and configuration files record user-configured arguments, training state history, the current training step, and the current learning rate. Thus, these files must be copied from the most recent checkpoint to assemble a new "Frankenstein" checkpoint while preserving training continuity and effectiveness. LLMTailor also provides an autonomous copy of these configuration files in the latest checkpoint that we create.

5 Experiments

In this section, we describe the experimental setup, models, datasets, baselines, and give 2 use case for our LLMTailor.

5.1 Experimental Setting

We conducted all real-world experiments on an 8-GPU cluster featuring NVIDIA A100 80GB GPUs and two AMD EPYC 7713 CPUs. The file system that we use is a Lustre file system mounted via an InfiniBand network. The used software versions are CUDA 12.8, DeepSpeed v0.17.2, and PyTorch 2.7.1. To address GPU memory constraints and train larger models, we employed DeepSpeed ZERO Stage-3 optimization, with the default optimizer being AdamW [16].

We evaluate LLMTailor using popular open-source LLMs, namely Llama-3.2-1B, Llama-3.1-8B [2], and Qwen-2.5-7B [26]. All experiments employ a sequence length of 2048. To quantify the reduction in checkpoint overhead in post-training, we test LLMTailor on two representative tasks: continual pre-training (CPT) [12] and supervised fine-tuning (SFT) [4]. Because post-training typically adapts a general-purpose LLM to specialized knowledge or capabilities, we select two medical datasets: (i) PubMed-Summarization, a plain-text corpus for CPT [3], and (ii) MedQA [15], a structured question-answering dataset for SFT. Each task is trained for one epoch. For PubMed-Summarization, we use a micro-batch size of 4 with 2 gradient-accumulation steps; for MedQA, we use a micro-batch size of 2 with 2 gradient-accumulation steps.

To assess model quality after recovery, we evaluate it on five benchmarks spanning three categories: medical expertise, general knowledge, and reasoning. Our goal is not to improve absolute

Model	Final train loss	Final eval loss
Qwen2.5-7B (After SFT)	1.58	1.60
Parity merge (start from 400)	1.58	1.60

(a) Qwen2.5-7B in a SFT task.

Model	Final train loss	Final eval loss
Llama3.1-8B (After CPT)	1.58	1.58
Filtered Layers (start from 1000)	1.58	1.58

(b) Llama3.1-8B in a CPT task.

Table 1: Comparison of training loss between original checkpoint and new checkpoint created by parity.

Task	Model	MMLU	MMLU_med	MedMCQA	MedQA	PubMedQA
SFT	Qwen2.5-7B	73.14	89.00	60.75	64.02	75.20
	parity-400	72.89	87.00	60.58	64.10	76.20
CPT	Llama3.1-8B	60.00	75.00	53.10	55.15	77.20
	parity-1000	60.03	72.00	53.12	54.36	76.60

Table 2: Zero-shot Benchmark Evaluation Results of resumed training in use case 1. (Question-Answer, higher is better)

scores but to verify that LLMTailor does not degrade performance in the post-training context. Consequently, we do not perform dataset deduplication before evaluation. Since no prior work has explored partial checkpointing, and partial checkpointing mechanisms can also be combined with prior work on I/O optimization and in-memory techniques, as the approaches are not mutually exclusive, we establish our baseline using the default checkpointing mechanism provided by the *transformers* library. Specifically, we adopt fixed checkpoint intervals of every 50 steps for the SFT task and every 100 steps for the CPT task, and then compare the time and storage overhead of our method against this baseline.

5.2 Use Case 1: Merge Checkpoints by Parity

The first use case that we decide to test is to merge the odd layers and the embed_token layer from the previous checkpoint, and the even layers and the lm_head layer from the current checkpoint. Considering the layer-wise structure of LLM is sequential, in order to avoid the split of the front and back layers of the checkpoint, we use this method to reconstruct a new one for resuming. Checkpointing only half of the complete checkpoint each time can reduce the storage overhead by almost half. Table 3 lists the exact size of each checkpoint.

Model	Type	Total CKPT size (G)	The proportion of checkpoint time(%)
Llama3.1-8B	Total	1799.52	4.99
	Parity	899.76	3.03
Qwen2.5-7B	Total	1811.52	20.63
	Parity	905.76	12.76

Table 3: Comparison between complete checkpoint and partial checkpoint in parity checkpoint

In Table 3, the proportion of checkpoint time means the ratio of checkpointing time and the end-to-end training time. These results suggest that in this use case, we can reduce the storage overhead by approximately 50% and checkpointing time overhead by 40%.

We then resume training using the new checkpoints generated by our LLMTailor framework. Table 1 compares the resumed training

process with that of the original model. The results show that both the training loss and the evaluation loss at the final step match those of the original training trajectory. This demonstrates the correctness of the checkpoint merging in this use case.

To assess the quality of the model after recovery and continued training, we evaluate the final, fully trained models on five benchmarks spanning three domains: medical expertise, general knowledge, and reasoning. Table 2 reports these results, where higher scores indicate better performance. The top results for each benchmark are highlighted. Here, the absolute score might not be the most important thing. Instead, it is crucial to verify that the Frankenstein model does not significantly degrade performance compared to the original model, which never experiences any failures or recoveries. Results show that our method can preserve model quality after recovering from the merged checkpoints. Therefore, it offers the possibility that future checkpointing systems could incorporate more precise mechanisms that allow for a slight trade-off in model performance to better balance overall model quality with the reduction in overhead.

5.3 Use Case 2: Merge Checkpoints by Filtering

Previous work shows that the first several layers and the last two layers of the model have a greater impact on model reasoning [11]. Based on that, we decide to checkpoint by filtering only the first and the last 2 layers each time, and checkpoint half of the other layers less often *every5 × original_interval*-to- to reduce more overhead. Table 6 lists the exact size of each checkpoint.

Here, we also set the default checkpoint mechanism as a baseline and measure the proportion of time spent on checkpoints. In this scenario, we can reduce the checkpoint time ratio to up to $2.8 \times$ for the Qwen2.5-7B model, and $4.3 \times$ storage overhead for the Llama3.1-8B model. We compare the size of the checkpoint with the default checkpoint method. Then, we use the new checkpoint created by our LLMTailor to resume training. Table 4 shows the results of their continued training compared to the original model.

We then resume training using the new checkpoints generated by our LLMTailor framework. Table 4 compares the resumed training process with that of the original model. Since lower loss indicates better performance, the results show that both the training loss

Model	Final train loss	Final eval loss
Qwen2.5-7B (After SFT)	1.58	1.60
Filtered Layers (start from 400)	1.60	1.62

(a) Qwen2.5-7B in an SFT task.

Model	Final train loss	Final eval loss
Llama3.1-8B (After CPT)	1.58	1.58
Filtered Layers (start from 1000)	1.59	1.59

(b) Llama3.1-8B in a CPT task.

Table 4: Comparison of training loss between original checkpoint and new checkpoint created by the impact of layers.

Task	Model	MMLU	MMLU_med	MedMCQA	MedQA	PubMedQA
SFT	Qwen2.5-7B	73.14	89.00	60.75	64.02	75.20
	filter-400	71.64	84.00	59.50	62.06	75.60
CPT	Llama3.1-8B	60.00	75.00	53.10	55.15	77.20
	filter-1000	62.06	77.00	53.45	54.91	78.00

Table 5: Zero-shot Benchmark Evaluation Results of resumed training in Use case2. (Question-Answer, higher is better)

Model	Type	Total CKPT size (G)	The proportion of checkpoint time(%)
Llama3.1-8B	Total	1799.52	4.99
	Filtered	420	1.66
Qwen2.5-7B	Total	1811.52	20.63
	Filtered	434.56	7.26

Table 6: Comparison between complete checkpoint and partial checkpoint in filtered checkpoint

Model Name	Checkpoint Size (G)	Total layers	CKPTs included	Time (s)
Llama3-1B	17.29	18	Baseline: 1	0.80
			2	117
			parity (2)	233.6
			8	60.4
Llama3-8B	112.47	35	18	62.5
			Baseline: 1	16.8
			2	332.4
			parity (2)	1027.5
			8	279.2
			35	264.3

Table 7: Loading time for different checkpoints

and the evaluation loss at the final step decrease slightly. This demonstrates that performance could degrade if we focus solely on reducing overhead at the expense of preserving the original functionality of checkpoints.

In the benchmark evaluation, Table 5 presents the results, where higher scores indicate better performance, and the top score for each benchmark is highlighted. In the SFT task, the newly created Qwen2.5 checkpoint performs noticeably worse than the default checkpoint, whereas in the CPT task, the composite Llama3 model noticeably outperforms the default model. This suggests that the inherent robustness of LLMs can, to some extent, support our concept of partial checkpointing. These relatively strong results from the rule-based partial checkpointing mechanism suggest that future systems employing more dynamic strategies in deciding which components to checkpoint and when are likely to achieve even better performance and greater robustness.

5.4 Checkpoint Overhead

We evaluate LLMTailor for splitting and merging different LLMs on both CPT and SFT tasks. The checkpoints we merge are partial. For the baseline, the reported time corresponds only to resuming a checkpoint, whereas for the other checkpoints, the reported time also includes the additional overhead of running LLMTailor. Therefore, in Table 7, we present the measured merging overhead from different perspectives.

Unless otherwise specified, checkpoints are loaded in a straightforward manner; for example, layers(1–16) are loaded from checkpoint-100, layers (17–32) from checkpoint-200, and so on. When we refer to “parity,” however, checkpoints are loaded multiple times

in an interleaved fashion: first, layer(1) from checkpoint-100 and layer(2) from checkpoint-200; then layer(3) from checkpoint-100 and layer(4) from checkpoint-200, and so forth. Even with only two checkpoints, this process still requires loading and discarding them N times, where N is the total number of layers. This approach incurs substantial overhead because the optimizer state can only be accessed after the checkpoint is fully loaded, with no possibility of lazy loading, as in the case of model weights. From this, we observe that the time overhead in LLMTailor is determined by: i) the loaded checkpoint size; ii) the number of loaded checkpoints; iii) the method we load the layers; iv) the number of total layers. Compared with the total training time, which can span several hours or even days, this overhead is relatively small and thus acceptable. We also observe that, although checkpoints must be loaded from N different files, each checkpoint contains only a single layer. As a result, the loading process is relatively fast. This observation suggests that the overhead of LLMTailor could be significantly reduced once a layer-wise checkpointing system is adopted.

6 Related Work

6.1 Checkpointing in Deep Learning.

Checkpoint techniques have been widely explored and utilized in many deep learning frameworks, such as PyTorch [22], TensorFlow [1]. However, default checkpoint systems will introduce significant overhead and stalls during the training stage, especially for large models. To address this challenge, many works have proposed different optimizations. Some works focus on optimizing the

checkpoint system, such as CheckFreq [20], which dynamically adjusts the checkpointing frequency to reduce I/O; Check-N-Run [8] compresses checkpoints with lossy schemes to reduce both I/O and required storage; [18, 25] recovers only checkpoints on GPUs that failed. Gemini [29] introduces in-memory checkpoint protection to avoid stalls and reduce recovery time by using high-bandwidth CPU memory, while Just-in-Time checkpointing [13] utilizes state redundancy in data parallel replicas of large deep learning jobs for efficient run-time checkpointing. In addition, Datastate-LLM [19] uses a lazy asynchronous multi-level approach, optimizing the pipeline of the checkpoint system to reduce overhead.

However, unlike all these methods, our approach focuses on identifying the non-uniform weight updates in LLM training and do layer-wise checkpointing to reduce the overhead, so that we can further reduce checkpoint size while utilizing current optimizations.

6.2 Model Merging

Researchers also disassemble and merge models. For instance, model merging is widely studied as an effective training-free method to combine the capabilities of fine-tuned large language models [30]. Yadav et al. [31] further help solve conflicts between different task vectors through norm-based sparsification and consensus on the signs of weights. Yu et al. [32] showed that LLMs are highly robust to sparsifying task vectors and proposed DARE for merging LLMs with random sparsification. Recently, Online Merging Optimizers [17] online merge the gradients of the policy model with the delta parameters of the SFT model. Our experiments show that both the model and the optimizer in checkpoints can also benefit from model merging.

7 Conclusion

In this paper, we introduce LLMTailor, a tool that enables effective and lightweight layer-wise checkpoints split-and-merge for large models. By performing layer-wise checkpointing, our use cases show the potential of reducing total checkpoint size and time by at least 60% compared with existing solutions, while preserving final model quality. Although our prototype can only manipulate local checkpoints, the underlying design can be applied to other checkpointing frameworks, and the implementation of this is part of our future work.

Acknowledgements

We sincerely thank the anonymous reviewers for their valuable feedback. This work was supported in part by the National Science Foundation (NSF) under grants CNS-2008265 and CCF-2412345. This effort was also supported in part by the U.S. Department of Energy (DOE) through the Office of Advanced Scientific Computing Research's "Orchestration for Distributed & Data-Intensive Scientific Exploration" and the "Decentralized data mesh for autonomous materials synthesis" AT SCALE LDRD at Pacific Northwest National Laboratory. PNNL is operated by Battelle for the DOE under Contract DE-AC05-76RL01830.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) (OSDI'16). USENIX Association, USA, 265–283.
- [2] AI@Meta. 2024. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [3] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. 2018. A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Association for Computational Linguistics, New Orleans, Louisiana, 615–621. doi:10.18653/v1/N18-2097
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. doi:10.18653/v1/N19-1423
- [5] Chris Egersdoerfer, Philip Carns, Shane Snyder, Robert Ross, and Dong Dai. 2025. STELLAR: Storage Tuning Engine Leveraging LLM Autonomous Reasoning for High Performance Parallel File Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'25)*.
- [6] Chris Egersdoerfer, Arnav Sareen, Jean Luca Bez, Suren Byna, and Dong Dai. 2024. ION: Navigating the HPC I/O Optimization Journey using Large Language Models. In *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*. 86–92.
- [7] Chris Egersdoerfer, Arnav Sareen, Jean Luca Bez, Suren Byna, Dongkuan Xu, and Dong Dai. 2025. IOAgent: Democratizing Trustworthy HPC I/O Performance Diagnosis Capability via LLMs. In *Proceedings of the 39th IEEE International Parallel & Distributed Processing Symposium (IPDPS)*.
- [8] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. 2022. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 929–943. <https://www.usenix.org/conference/nsdi22/presentation/eisenman>
- [9] Swapnil Gandhi and Christos Kozyrakis. 2025. MoEtion: Efficient and Reliable Sparse Checkpointing for Mixture-of-Experts Models at Scale. arXiv:2412.15411 [cs.DC] <https://arxiv.org/abs/2412.15411>
- [10] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vladimir Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. 2024. Arcee's MergeKit: A Toolkit for Merging Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preotjiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 477–485. doi:10.18653/v1/2024.emnlp-industry.36
- [11] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Gloriosio, and Daniel A. Roberts. 2025. The Unreasonable Ineffectiveness of the Deeper Layers. arXiv:2403.17887 [cs.CL] <https://arxiv.org/abs/2403.17887>
- [12] Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. 2023. Continual Pre-Training of Large Language Models: How to (re)warm your model? arXiv:2308.04014 [cs.CL] <https://arxiv.org/abs/2308.04014>
- [13] Tanmaey Gupta, Sanjeev Krishnan, Rituraj Kumar, Abhishek Vijeev, Bhargav Gulavani, Nipun Kwatra, Ramachandran Ramjee, and Muthian Sivathanu. 2024. Just-In-Time Checkpointing: Low Cost Error Recovery from Deep Learning Training Failures. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (EuroSys '24). Association for Computing Machinery, New York, NY, USA, 1110–1125. doi:10.1145/3627703.3650085
- [14] Ganesh Jawahar, Benoit Sagot, and Djamel Seddah. 2019. What Does BERT Learn about the Structure of Language?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Anna Korhonen, David Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, Florence, Italy, 3651–3657. doi:10.18653/v1/P19-1356
- [15] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences* 11, 14 (2021), 6421.
- [16] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. arXiv:1711.05101 [cs.LG] <https://arxiv.org/abs/1711.05101>
- [17] Keming Lu, Bowen Yu, Fei Huang, Yang Fan, Runji Lin, and Chang Zhou. 2024. Online Merging Optimizers for Boosting Rewards and Mitigating Tax in Alignment. arXiv:2405.17931 [cs.CL] <https://arxiv.org/abs/2405.17931>

- [18] Kiwan Maeng, Shivam Bharuka, Isabel Gao, Mark C. Jeffrey, Vikram Saraph, Bor-Yiing Su, Caroline Trippel, Jiyan Yang, Mike Rabbat, Brandon Lucia, and Carole-Jean Wu. 2020. CPR: Understanding and Improving Failure Tolerant Training for Deep Learning Recommendation with Partial Recovery. arXiv:2011.02999 [cs.LG] <https://arxiv.org/abs/2011.02999>
- [19] Avinash Maurya, Robert Underwood, M. Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (Pisa, Italy) (HPDC '24)*. Association for Computing Machinery, New York, NY, USA, 227–239. doi:10.1145/3625549.3658685
- [20] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 203–216. <https://www.usenix.org/conference/fast21/presentation/mohan>
- [21] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025), 1–72.
- [22] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
- [23] Jason Phang, Haokun Liu, and Samuel R. Bowman. 2021. Fine-Tuned Transformers Show Clusters of Similar Representations Across Layers. In *Proceedings of the Fourth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, Jasmijn Bastings, Yonatan Belinkov, Emmanuel Dupoux, Mario Giulianelli, Dieuwke Hupkes, Yuval Pinter, and Hassan Sajjad (Eds.). Association for Computational Linguistics, Punta Cana, Dominican Republic, 529–538. doi:10.18653/v1/2021.blackboxnlp-1.42
- [24] Ofir Press and Lior Wolf. 2017. Using the Output Embedding to Improve Language Models. arXiv:1608.05859 [cs.CL] <https://arxiv.org/abs/1608.05859>
- [25] Aurick Qiao, Bryon Aragam, Bingjing Zhang, and Eric Xing. 2019. Fault Tolerance in Iterative-Convergent Machine Learning. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 5220–5230. <https://proceedings.mlr.press/v97/qiao19a.html>
- [26] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuzhong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- [27] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Atlanta, Georgia) (SC '20)*. IEEE Press, Article 20, 16 pages.
- [28] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 3505–3506. doi:10.1145/3394486.3406703
- [29] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. 2023. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 364–381. doi:10.1145/3600006.3613145
- [30] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. arXiv:2203.05482 [cs.LG] <https://arxiv.org/abs/2203.05482>
- [31] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. 2023. TIES-MERGING: resolving interference when merging models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 310, 23 pages.
- [32] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. 2024. Language models are super mario: absorbing abilities from homologous models as a free lunch. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 2382, 21 pages.
- [33] Yefan Zhou, Tianyu Pang, Keqin Liu, Charles H. Martin, Michael W. Mahoney, and Yaoqing Yang. 2023. Temperature balancing, layer-wise weight analysis, and

neural network training. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2775, 31 pages.

A Overview of Contributions and Artifacts

A.1 Paper’s Main Contributions

- C₁ We present LLMTailor, an effective tool that assembles a resumable checkpoint from parts of multiple checkpoints while maintaining the model performance.
- C₂ We demonstrate that partial checkpointing with LLMTailor can substantially lower storage requirements and checkpoint time in LLM training.
- C₃ We show that LLMTailor incurs only a small amount of overhead when resuming training from a merged checkpoint.

A.2 Computational Artifacts

A₁ <https://doi.org/10.5281/zenodo.16909083>

Artifact ID	Contributions Supported	Related Paper Elements
A ₁	C ₁ , C ₂ , C ₃	Tables 1-7 Figure 3

B Artifact Identification

B.1 Computational Artifact A₁

Relation To Contributions

The artifact provides the implementation of the methods and ideas presented in the paper. Our LLMTailor, included as the artifact, supports fine-grained manipulation of model and optimizer states across checkpoints, selective merging part of the checkpoints, and reconstruction of training state. It is also the key to realizing the proposed checkpoint strategies as it enables partial layer saving.

Expected Results

- (1) **Flexibility in Checkpoint Composition** By using LLM-Tailor, we are able to assemble “Frankenstein” checkpoints. It selects and mixes layers from different checkpoints and shows both functional correctness (the model runs and trains as expected) and practical utility (supporting adaptive failure recovery or analysis).
- (2) **Reduced Checkpointing Overhead** The experiments will show that LLMTailor enables selective, layer-level checkpointing which can reduce the size and time of saved checkpoints. Compared to traditional full-model checkpoints, the system achieves significant I/O savings and reduced storage usage without sacrificing recoverability.
- (3) **Faithful Model Recovery and Continuity** By merging layers and optimizer states from multiple checkpoints, use case 1 (merge by parity) demonstrates that training can be faithfully resumed from partial checkpoints. The outcome to expect is a recovery trajectory that closely matches (or even exactly overlays) the trajectory from full checkpointing baselines. While in use case 2 (filter layers), the recovered training will have a bias with the original checkpoint.

Expected Reproduction Time (in Minutes)

The expected computational time of this artifact on GPU is 60 min.

Artifact Setup (incl. Inputs)

Hardware. One 8× A100(40GB-memory) node.

CPU: At least 64 cores.

Memory: At least 200 GB.

Storage: Depending on the model and training epochs, recommend at least 350 GB for a 7B model and 700 GB for 14B model.

Software. The used software are CUDA-12.8, DeepSpeed-v0.17.2, PyTorch-2.7.1, transformers-4.55.0, pydantic-2.9.2, flash_attn-2.6.3. The used models are Llama-3.2-1B, Llama-3.1-8B, and Qwen-2.5-7B. (<https://huggingface.co/meta-llama/Llama-3.2-1B>), (<https://huggingface.co/meta-llama/Llama-3.1-8B>), (<https://huggingface.co/Qwen/Qwen2.5-7B>).

Datasets / Inputs. We use two medical datasets:

- (i) PubMed-Summarization (<https://huggingface.co/datasets/ccdv-pubmed-summarization>)
- (ii) MedQA (<https://huggingface.co/datasets/Malikeh1375/medical-question-answering-datasets>)

Installation and Deployment. First, using anaconda to create an environment using python version 3.11. Then using git to clone our artifacts and using 'pip install -r requirements.txt' to install our dependencies.

The examples are in the folder '/example'. Then, modifying the YAML file to whatever you like. Next, modifying the configuration in the top of this start_merge.py file. (e.g. CHECKPOINT_PATH). And finally, run these steps in example.ipynb.

For the benchmark running tasks, we use the open source project called lm-evaluation-harness (<https://github.com/EleutherAI/lm-evaluation-harness>). Please follow the instructions of this project to install and use.

Artifact Execution

The workflow consists of 3 tasks:

- (1) T_1 . Run an LLM training job that generates multiple checkpoints. Our package can be used as a submodule within a partial checkpointing framework, and is fully compatible with checkpoints produced in this form.
- (2) T_2 . To configure and start our program, the user should provide: the path to the previous checkpoints, the output path for the new "Frankenstein" checkpoint, the number of GPUs, the failure step number, and the number of hidden layers in the training model. If partial checkpoints are used, the user may also provide the JSON file generated by the partial checkpointing system. In this case, our tool will automatically generate a corresponding YAML file. Otherwise, the user can manually write a YAML file specifying the layers to be merged. Finally, LLMTailor will select layers from different checkpoints and assemble them into a complete new checkpoint.
- (3) T_3 . Use the path of the newly generated checkpoint to resume training. Observe whether the loss curves align with

those of uninterrupted training, thereby confirming recovery from failure. For the final stage, evaluate the model performance on the benchmarks mentioned in the paper to validate correctness and consistency.

Artifact Analysis (incl. Outputs)

Running Task T_1 will generate multiple checkpoints together with an optional JSON file that records the partial checkpointing decisions. Different layers are checkpointed at different steps, and the detailed information is logged in the JSON files. By comparing the size of these checkpoints, one can verify that when only a subset of layers is saved, the resulting checkpoint is smaller than a full checkpoint.

Task T_2 produces a complete checkpoint folder. Users can confirm correctness by comparing its size and file structure against a standard full checkpoint generated without selective saving.

Executing Task T_3 demonstrates recovery from a simulated failure. Training should resume exactly at the failure step, and the reported loss and downstream performance metrics will confirm that recovery is almost consistent with uninterrupted training.